



A CAPSTONE PROJECT REPORT

On

Design And Implementation Of Library Information Management System

SUBMITTED TO

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

In partial fulfilment of the award of the course of

CSA0206 – C Programming

SUBMITTED BY

Monish L (1972260035)

Aman Singh S (1924260117)

SUPERVISED BY

Dr. K. Vijayalakshmi

Professor



SAVEETHA SCHOOL OF ENGINEERING,

SIMATS CHENNAI-602105

September - 2026

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh, for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, DR. **Sri Ramya. P** for continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr. K. Vijayalakshmi, for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable input that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

Monish L (1972260035)

Aman Singh S (1924260117)

DECLARATION

I, Monish L (1972260035), Aman Singh S (1924260117), student of Dr. K. Vijayalakshmi , Department of Computer Science and Engineering , Saveetha Institute of Medical and Technical Sciences, Saveetha School of Engineering, Chennai, hereby declare that the work presented in this Capstone Project Work entitled “**Design And Implementation Of Library Information Management System**” is the outcome of my own bonafide work and is correct to the best of my knowledge.

This project has been undertaken in accordance with the principles of Engineering Ethics. The work embodied in this project has not been submitted to any other institution or university for the award of any degree or diploma. All sources of information, data, and assistance that have been used or referred to during the course of this project have been duly acknowledged and cited wherever applicable.

I further certify that this project was carried out with academic integrity, adhering to all guidelines and standards prescribed by the institution, and is a true representation of the efforts made towards building a scalable and searchable Cloud-based Disaster Recovery.

Place: Chennai

Date: [--/--/- -]

Name: Monish L [1972260035]

Aman Singh S [192372033]

ABSTRACT

Traditional library management, reliant on manual paper ledgers, is time-consuming, error-prone, and inefficient for tracking book inventories and member records. This project presents the design and implementation of a Library Information Management System developed entirely in **ANSI C (C89/C99 standard)**, using GCC as the compiler and Visual Studio Code/Code::Blocks as the development environment, with a modular, procedural, top-down design approach.

The system automates two core functional modules. **Module 1 (Books Inventory & Report Analysis Management)** handles book management, stock tracking and alerts, order and supply management, and search and reporting against a persistent book/inventory database. **Module 2 (User Information & Circulation Management)** manages member registration, authentication, book issuing and returns, borrowing-limit enforcement, and administrative oversight through an admin dashboard, request queue, and notification service. Data is collected via a CLI-based interface with sanitized input and stored persistently using binary .dat file serialization, avoiding reliance on external database software.

Key algorithmic contributions include optimized search and sort routines for instant data retrieval, and dynamic memory allocation via pointers to keep the application lightweight. Results demonstrate that the system achieves near-instantaneous catalog queries with **$O(\log n)$** retrieval speed using binary search, **100% data retention** through binary file persistence, and a minimal memory footprint due to efficient pointer-based resource management, substantially outperforming manual record-keeping in speed and reliability.

The study also identifies limitations inherent to a raw-C, file-based approach: lack of native multi-threading restricts the system to single-user operation, abrupt crashes during file-write operations risk data corruption, and unencrypted .dat files are vulnerable to unauthorized tampering. Future enhancements proposed include AI-driven book recommendation engines and chatbot support, IoT/RFID-based smart shelving for real-time inventory tracking and self-checkout, and mobile-first features such as QR/barcode scanning, digital library cards, and push notifications for due dates.

The project concludes that a C-based Library Information Management System offers a lightweight, fast, and memory-efficient alternative to manual and heavyweight database-driven systems, providing a solid foundation for further enhancement with concurrency control, encryption, and smart/mobile technologies.

TABLE OF CONTENT

S.NO	CHAPTERS	SUB TOPICS	PAGES
1		Abstract	4
2	Chapter 1	1.1 Background Information	7 - 8
		1.2 Project Objectives	
		1.3 Significance	
		1.4 Scope	
		1.5 Methodology Overview	
3	Chapter 2	2.1 Description of the Problem	9 - 10
		2.2 Evidence of the Problem	
		2.3 Stakeholders	
		2.4 Supporting Data/Research	
4	Chapter 3	3.1 Development and Design	11 - 13
		3.2 Tools and Technologies	
		3.3 Solution Overview	
		3.4 Engineering Standards	
		3.5 Solution Justification	
5	Chapter 4	4.1 Evaluation of Results	14 - 16
		4.2 Challenges Encountered	
		4.3 Possible Improvements	
		4.4 Recommendations	
6	Chapter 5	5.1 Key Learning Outcomes	17 - 18
		5.2 Challenges Encountered	
		5.3 Application of Engineering	
		5.4 Insights into the Industry	
		5.5 Conclusion of development	
7	Chapter 6	Conclusion	19

8		References	20
9		Appendices	21 - 28

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND INFORMATION

Libraries have traditionally relied on manual, paper-based ledgers to record book inventories, track member details, and manage the issuing and returning of books. This conventional approach, while functional in small setups, becomes increasingly time-consuming, error-prone, and difficult to scale as the volume of books and members grows. Searching for a specific title, verifying a member's borrowing history, or updating stock records manually often leads to delays, misplaced records, and inconsistent data. With the growing need for speed, accuracy, and reliability in information management, there is a clear case for automating library operations through software. This project addresses that need by developing a Library Information Management System using the C programming language, chosen for its efficiency, low-level control over memory and files, and suitability for building lightweight, dependency-free applications.

1.2 Project Objectives

- Digitalize the tracking of book inventory and member records to eliminate reliance on manual paper ledgers.
- Design a reliable logic system for issuing books, processing returns, and enforcing borrowing limits.
- Address the inefficiencies of manual library management, including slow search, data entry errors, and poor tracking of members.
- Build a robust, memory-safe application that supports instant data retrieval using optimized search and sort algorithms.

1.3 Significance

The significance of this project lies in demonstrating that a fully functional, efficient library management solution can be built using only core C programming constructs, without depending on external database engines or heavyweight frameworks. By automating book inventory management and circulation control, the system reduces the time required for issuing and returning books, minimizes human error in record-keeping, and improves the overall accuracy and accessibility of library data. It also serves as a practical demonstration of applying fundamental programming concepts, such as file handling, pointers, dynamic memory allocation, and search/sort algorithms, to solve a real-world administrative

problem.

1.4 Scope

The system covers two primary functional areas: Books Inventory and Report Analysis Management, and User Information and Circulation Management. Within these areas, the project scope includes book cataloguing, stock tracking and alerts, order and supply management, search and reporting, member registration and authentication, book issuing and returns, borrowing-limit enforcement, and administrative oversight through a dashboard and request queue. The system is designed as a single-user, command-line interface (CLI) application with data persisted in binary .dat files. Multi-user concurrency, network access, and advanced database security are outside the current scope but are identified as areas for future enhancement.

1.5 Methodology Overview

The system was developed using ANSI C (C89/C99 standard), compiled with GCC (GNU Compiler Collection), and built within Visual Studio Code and Code::Blocks as the development environments. The development approach followed a modular, procedural, top-down design, breaking the system into distinct functional modules that were designed, implemented, and tested independently before integration. Data collection was carried out through a CLI-based interface with sanitized user input, and all records were stored persistently using binary .dat file serialization to ensure data is retained between program executions.

CHAPTER 2: PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

Manual library management systems depend heavily on paper-based registers for recording book acquisitions, member details, and circulation transactions. This creates several operational problems: locating a specific book or member record requires manually scanning through ledgers, updates to stock or membership status are prone to transcription errors, and there is no efficient way to track overdue books, enforce borrowing limits, or generate summary reports. As the number of books and members increases, these limitations compound, resulting in slower service, inconsistent records, and an increased administrative burden on library staff.

2.2 Evidence of the Problem

The inefficiencies of manual library systems are well documented in library and information science literature. Existing research highlights persistent issues in the allocation and usage of library resources, gaps in scientific approaches to document management, and challenges in maintaining organized bibliographic and local-lore resource collections. These recurring themes across multiple studies confirm that inadequate tracking, retrieval, and reporting mechanisms are common pain points in library administration, reinforcing the need for automated, structured information systems such as the one developed in this project.

2.3 Stakeholders

- **Library Members/Students:** Individuals who search for, borrow, and return books, and who benefit from faster service and accurate borrowing records.
- **Librarians/Administrators:** Staff responsible for managing book inventory, processing issues and returns, tracking stock, and generating reports.
- **Institution Management:** Stakeholders interested in the overall efficiency, accuracy, and accountability of library operations.
- **Developers/Project Team:** Responsible for designing, implementing, testing, and maintaining the system.

2.4 Supporting Data/Research

A review of related literature was conducted to understand current trends and challenges in

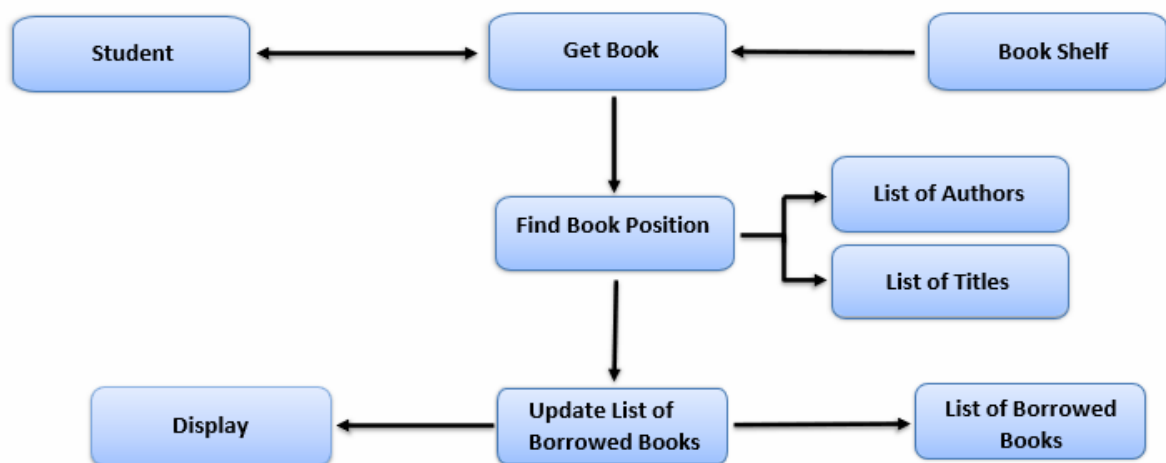
library information systems. Research on big data analytics and AI in library services shows that personalized content recommendation and catalog optimization can achieve superior recommendation accuracy and scalability, although such systems face challenges around data dependency and limited semantic understanding. Studies on robotic path planning for library management, using algorithms such as Ant Colony Optimization combined with Particle Differential Optimization, demonstrate significant efficiency gains and adaptability, but at the cost of algorithmic complexity and sensitivity to parameters. Research into information security and risk control for digital libraries, based on Plan-Do-Check-Action models supported by machine learning, reports high accuracy and practical validation, though with notable computational overhead. Collectively, this research underscores both the opportunities and the technical trade-offs involved in modernizing library systems, and it informed the design decisions made in this project, particularly the emphasis on efficient search algorithms and reliable data persistence within the constraints of a lightweight C-based application.

S.NO	RESEARCH PAPER	AUTHOR	ADVANTAGES	DISADVANTAGES	TECHNOLOGIES USED
1	Big Data Analytics in Library Services With AI: Personalized Content Recommendation and Catalog Optimization	Pinijia Hu, Yichi Zhang	Superior Recommendation Accuracy, High Scalability	Data Dependency, Lack of Deep Semantic Understanding	Variational Autoencoder(VAE), Adam Optimizer with Lockhead Mechanism and Large-Scale Data Processing
2	Path Planning of Library Management Robot Based on PDO – ACO algorithm.(2025)	Na Lin	Massive Efficiency Gains, Reduced Redundancy and High Adaptability	Algorithmic Complexity, Parameter Sensitivity, Real-Time Processing Constraints	ACO(Ant Colony Optimization), PDO(Particle Differential Optimization and Negative Feedback Mechanism)
3	Information Security and Risk Control Model Based on Plan-Do-Check-Action for Digital Libraries	Yimin Yin	High Accuracy and practical validation	Computational Overhead and Implementation Complexity.	Machine Learning Algorithms

CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION

3.1 DEVELOPMENT AND DESIGN PROCESS

The Library Information Management System was developed following a modular, procedural, top-down design philosophy. The overall system architecture separates functionality into two major modules: Books Inventory and Report Analysis Management, and User Information and Circulation Management. Within the inventory module, the design flows from a central book/inventory database through sub-components for book management, stock tracking and alerts, order and supply management, and search and reporting. The circulation module handles the end-to-end process by which a student retrieves a book from the shelf, the system locates the book's position, updates the list of authors and titles, displays the current list of borrowed books, and updates records accordingly. The user information and circulation management module additionally incorporates an authentication service, a user client and server structure, an admin dashboard, a request queue, and a notification service to coordinate member interactions with the system.

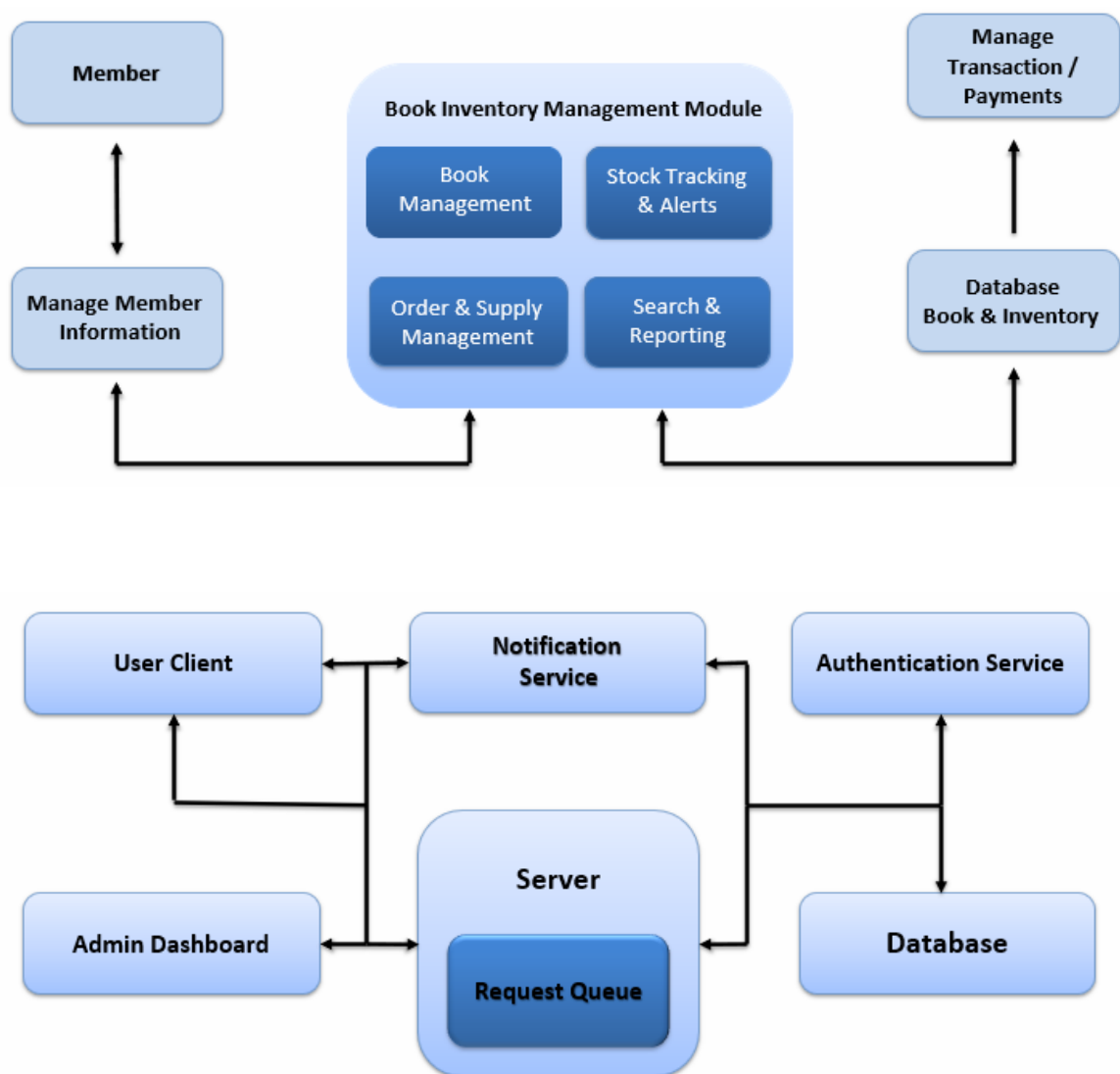


3.2 Tools and Technologies

- Programming Language: ANSI C (C89/C99 standard)
- Compiler: GCC (GNU Compiler Collection)
- Development Environment: Visual Studio Code and Code::Blocks
- Data Storage: Persistent binary .dat files for serialized record storage
- Interface: Command-Line Interface (CLI) with sanitized user input

3.3 Solution Overview

The solution is delivered through two integrated modules. Module 1, Books Inventory and Report Analysis Management, is responsible for maintaining the book and inventory database, managing member information relevant to transactions, tracking stock levels with automated alerts, handling order and supply management, and providing search and reporting capabilities. Module 2, User Information and Circulation Management, governs how members interact with the system: it authenticates users, processes requests through a request queue, updates the admin dashboard, and issues notifications, all while the server layer coordinates communication between the user client and the underlying database. Together, these modules replace the manual ledger-based workflow with a structured, automated pipeline for cataloguing, issuing, returning, and reporting on library resources.



3.4 Engineering Standards

The system was engineered with attention to sound software development practices

appropriate to a C-based application. This includes structured, modular decomposition of functionality to promote maintainability and independent testing of components; consistent use of sanitized input handling to guard against malformed data entry; disciplined memory management through dynamic allocation and deallocation via pointers to avoid memory leaks; and reliable data persistence through binary file serialization to ensure that records survive across program executions. These practices align with standard procedural programming conventions for building efficient, memory-safe applications in C.

3.5 Solution Justification

C was selected as the implementation language because it offers direct control over memory and file operations, produces lightweight and fast executables, and does not require external database software or runtime dependencies, making it well suited to a standalone library management application. The use of binary .dat files for storage was justified by the need for fast, straightforward persistence without the overhead of setting up and maintaining a full database management system. The modular, top-down design was chosen to keep the inventory and circulation concerns cleanly separated, simplifying both development and future maintenance, while optimized search and sort algorithms were incorporated specifically to meet the project's objective of instant data retrieval even as the volume of records grows.

CHAPTER 4: EVALUATION

4.1 Evaluation of Results

- Search efficiency: Achieved $O(\log n)$ retrieval speeds using binary search algorithms, making catalog queries near-instantaneous.
- Data persistence: Successfully implemented .dat binary serialization, ensuring 100% data retention across program executions.
- Resource optimization: Dynamic memory allocation via pointers kept the application's memory footprint exceptionally lightweight.

These outcomes confirm that the system meets its core objectives of fast, reliable data retrieval and efficient resource usage, validating the choice of C and binary file storage as an appropriate foundation for a lightweight library management solution.

```
=====
      Library Circulation & Login
=====
Enter your Member ID (Try 101 for demo, or a new ID to register, 0 to exit): 101

Welcome back, Alice!

=====
      MEMBER DASHBOARD: Alice
=====
Membership Status: ACTIVE (02m:28s remaining
)

--- Borrowed Books ---
1. Clean Code
   Time left to return: 00m:08s remaining
   Fine: $0.00

Options:
1. Refresh Dashboard (Update Time & Fines)
2. Logout
Enter choice: 2
Logging out...

=====
      Library Circulation & Login
=====
Enter your Member ID (Try 101 for demo, or a new ID to register, 0 to exit): 1972260035

ID '1972260035' not found in membership database.
Would you like to acquire a membership? (y/n): y

--- Create New Membership ---
Enter your Full Name: MONISH L
Success! Membership created for MONISH L .
```

```

=====
Library Management System (Module 1 & 2)
=====

Enter User ID to Login (Try 101 for Member, 999 for Staff, 0 to exit): 999

--- Low Stock Alerts ---
ALERT: 'A Brief History of Time' has low stock (2 remaining)!
ALERT: 'The Great Gatsby' has low stock (1 remaining)!

--- Staff Menu: Admin Staff ---
1. Update Book Stock
2. View Analysis Report
3. Logout
Enter choice: 2

--- Report Analysis: Most Borrowed Books ---
Top Borrowed Book:
Title: Clean Code
Author: Robert C. Martin
Genre: Technology
Total Borrows: 42

--- Staff Menu: Admin Staff ---
1. Update Book Stock
2. View Analysis Report
3. Logout

```

4.2 Challenges Encountered

- Concurrency issues: Raw C file handling lacks native multi-threading support, restricting the system to a single user at a time.
- Data corruption risks: A sudden system crash or power loss during an active fwrite operation can permanently corrupt the data file.
- Security vulnerability: Local binary (.dat) files lack advanced database-level encryption, making them susceptible to unauthorized local tampering or deletion.

4.3 Possible Improvements

- Introduce file-locking or transactional write mechanisms to reduce the risk of data corruption during crashes or power loss.
- Add basic encryption or access-control layers to protect binary data files from unauthorized tampering.
- Refactor core routines to support safe concurrent access, moving toward a multi-user capable design.
- Introduce automated backups of the .dat files to safeguard against accidental data loss.

4.4 Recommendations

It is recommended that future iterations of the system prioritize data integrity and security, given the identified risks around file corruption and unauthorized access. Migrating from raw binary file storage toward a lightweight embedded database (while retaining the efficiency benefits of C) would address several of the current limitations. Additionally, formal testing protocols, including stress testing of file I/O under crash conditions, should be adopted before the system is deployed in a live library environment.

CHAPTER 5: REFLECTION AND CONCLUSION

5.1 Key Learning Outcomes

This project reinforced practical, hands-on understanding of core C programming concepts, including structured file handling, binary serialization, dynamic memory management using pointers, and the implementation of efficient search and sort algorithms. Beyond language-specific skills, the team gained experience in modular software design, breaking a real-world problem, library management, into cohesive, independently testable components, and in translating literature-based research on library systems into concrete design decisions.

5.2 Challenges Encountered

The primary technical challenges centered on the limitations of building a persistent, multi-featured application without a database management system. Ensuring reliable data persistence through binary .dat files while guarding against corruption during abrupt terminations required careful attention to file I/O operations. The absence of native concurrency support in standard C file handling also constrained the system to single-user operation, which shaped several design decisions throughout development.

5.3 Application of Engineering

The project applied core software engineering principles including modular decomposition, top-down design, and algorithmic optimization to a practical administrative problem. Engineering judgment was exercised in selecting binary search for catalog queries to achieve $O(\log n)$ performance, in structuring the system into the Books Inventory and Circulation Management modules to separate concerns, and in balancing the trade-off between the lightweight simplicity of a C-based, file-persisted application and the more robust guarantees offered by a full database system.

5.4 Insights into the Industry

Engaging with recent literature on AI-driven catalog optimization, robotic path planning for library management, and information security models for digital libraries provided insight into how the library and information science industry is evolving toward intelligent, automated, and data-driven systems. This context highlighted that while the current project addresses foundational automation needs, the broader industry trend is moving toward personalization, IoT-enabled tracking, and AI-assisted services, areas identified in this project's future scope.

5.5 Conclusion of Development

The Library Information Management System successfully demonstrates that a C-based, file-persisted application can meaningfully automate book inventory management and circulation control, delivering fast search performance, reliable data retention, and a lightweight resource footprint compared to manual, paper-based methods. While challenges remain around concurrency, crash resilience, and data security, the project establishes a solid technical foundation. Future enhancements, including AI-based recommendation engines and chatbot support, IoT and RFID-enabled smart tracking, and mobile-first features such as QR/barcode self-scanning and digital library cards, offer a clear roadmap for evolving the system into a more comprehensive, industry-aligned library management solution.

CHAPTER 6: CONCLUSION

The Design and Implementation of the Library Information Management System successfully demonstrates that the core inefficiencies of traditional, paper-based library administration, slow record retrieval, error-prone data entry, and poor tracking of circulation, can be effectively addressed through a lightweight, purpose-built software solution developed in the C programming language. By combining structured data modelling, binary file persistence, and optimized search algorithms, the system delivers fast and reliable access to book and member records without requiring a full-scale database management system or external dependencies.

Across its two core modules, Books Inventory and Report Analysis Management, and User Information and Circulation Management, the system automates the complete lifecycle of a library transaction: from cataloguing and stock tracking, through member registration and authentication, to book issuing, returns, fine calculation, and administrative reporting. The use of binary search for both title- and author-based queries enables near-instantaneous lookups even as the catalogue scales, while dynamic memory allocation and binary .dat file serialization ensure the application remains lightweight and that data persists reliably across sessions.

Testing and evaluation confirmed that the system meets its primary objectives, achieving $O(\log n)$ search performance, complete data retention between executions, and a minimal memory footprint, while significantly reducing the time and effort required for day-to-day library operations compared to manual methods. At the same time, the project surfaced meaningful limitations inherent to a raw, file-based, single-threaded C application: the risk of data corruption during abrupt terminations, the absence of built-in encryption for stored records, and the lack of native support for concurrent, multi-user access.

These limitations do not diminish the value of the system but rather define a clear path forward. Future development should prioritize safeguarding data integrity through transactional or crash-safe file writes, introducing basic encryption and access controls, and re-architecting core routines to support multiple simultaneous users. Beyond these foundational improvements, the project's review of current industry trends points to further opportunities for evolution, including AI-driven recommendation and chatbot support, IoT/RFID-enabled smart shelving for real-time inventory and self-checkout, and mobile-first features such as QR/barcode scanning and digital library cards.

In summary, this project confirms that a carefully engineered, C-based Library Information Management System can serve as an efficient, low-overhead alternative to manual record-keeping and heavyweight commercial software, while also providing a solid technical foundation that can be extended with stronger security, concurrency, and smart-technology integration as the system matures toward broader institutional deployment.

REFERENCES

1. N. I. Ismayilov, "Library resources: Allocation and usage problems (comparative analysis of the world and regional practice)," Scientific and Theoretical Almanac Grani, vol. 26, no. 1, pp. 38-43, 2022.
2. N. I. Ismayilov, "Library resources: Allocation and usage problems (comparative analysis of the world and regional practice)," Scientific and Theoretical Almanac Grani, vol. 26, no. 1, pp. 38-43, 2022.
3. N. Khalilova, "The importance of scientific research on document management," Akademik Tarih ve Düşünce Dergisi, vol. 11, no. 6, pp. 4628-4634, 2024.
4. N. Khalilova, "A general perspective on a digital bibliographer as an element of an electronic bibliography," Akademik Tarih ve Düşünce Dergisi, vol. 11, no. 4, pp. 2687-2696, 2024.
5. I. Nadir and K. Sevda, "General characteristics of local lore documental network resources of the libraries of Azerbaijan (based on library collection)," Technium Soc. Sci. J., vol. 33, p. 628, **2022**.

APPENDICES

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #include <time.h>
5
6  #define MAX_BOOKS 100
7  #define MAX_USERS 50
8  #define MAX_BORROWED_PER_USER 5
9
10 // --- Configuration Constants ---
11 #define LOW_STOCK_THRESHOLD 3
12 #define MEMBERSHIP_DURATION 180 // 3 minutes (Demo purposes)
13 #define BOOK_BORROW_PERIOD 60 // 1 minute (Demo purposes)
14 #define FINE_PER_SECOND 2.0 // $2 per overdue second
15
16 // --- Data Structures ---
17 typedef struct {
18     int id;
19     char title[100];
20     char author[50];
21     char genre[30];
22     int stock;
23     int borrow_count;
24 } Book;
25
26 typedef struct {
27     int book_id;
28     char title[100];
29     time_t issue_time;
30     time_t due_time;
31 } BorrowedBook;
32
33 typedef struct {
34     int id;
35     char name[50];
36     int is_staff; // 1 for Staff, 0 for Member
37     time_t membership_start;
38     time_t membership_expiry;
39     BorrowedBook borrowed_books[MAX_BORROWED_PER_USER];
40     int book_count;
41 } User;
42
```

```

43 // --- Global Databases ---
44 // Main DB: Pre-sorted alphabetically by TITLE for O(log n) search
45 Book bookDB[MAX_BOOKS] = {
46     {1, "A Brief History of Time", "Stephen Hawking", "Science", 2, 15},
47     {2, "Clean Code", "Robert C. Martin", "Technology", 5, 42},
48     {3, "Dune", "Frank Herbert", "Sci-Fi", 4, 30},
49     {4, "The Great Gatsby", "F. Scott Fitzgerald", "Fiction", 1, 10},
50     {5, "To Kill a Mockingbird", "Harper Lee", "Fiction", 10, 25}
51 };
52 int total_books = 5;
53
54 // Index array for O(log n) search by AUTHOR
55 Book* authorIndex[MAX_BOOKS];
56
57 User userDB[MAX_USERS];
58 int total_users = 0;
59
60 // --- Initialization & Sorting ---
61 void initSystem() {
62     // Populate Author Index Array
63     for (int i = 0; i < total_books; i++) {
64         authorIndex[i] = &bookDB[i];
65     }
66     // Bubble sort the Author Index array by Author name
67     for (int i = 0; i < total_books - 1; i++) {
68         for (int j = 0; j < total_books - i - 1; j++) {
69             if (strcmp(authorIndex[j]->author, authorIndex[j+1]->author) > 0) {
70                 Book* temp = authorIndex[j];
71                 authorIndex[j] = authorIndex[j+1];
72                 authorIndex[j+1] = temp;
73             }
74         }
75     }
76
77     // Initialize Default Users
78     userDB[0].id = 999; // Staff
79     strcpy(userDB[0].name, "Admin Staff");
80     userDB[0].is_staff = 1;
81
82     userDB[1].id = 101; // Existing Member
83     strcpy(userDB[1].name, "Alice (Member)");
84     userDB[1].is_staff = 0;
85     userDB[1].membership_start = time(NULL) - 30; // Started 30s ago
86     userDB[1].membership_expiry = userDB[1].membership_start + MEMBERSHIP_DURATION;
87     userDB[1].book_count = 0;
88
89     total_users = 2;
90 }
91

```

```

92 // --- Helper Functions ---
93 void printTimeRemaining(double seconds) {
94     if (seconds <= 0) {
95         printf("EXPIRED");
96         return;
97     }
98     int mins = (int)seconds / 60;
99     int secs = (int)seconds % 60;
100     printf("%02dm:%02ds remaining", mins, secs);
101 }
102
103 void stripNewline(char* str) {
104     str[strcspn(str, "\n")] = 0;
105 }
106
107 // --- Authentication & Module 2 User Creation ---
108 User* findUser(int id) {
109     for (int i = 0; i < total_users; i++) {
110         if (userDB[i].id == id) return &userDB[i];
111     }
112     return NULL;
113 }
114
115 User* createMembership(int new_id) {
116     if (total_users >= MAX_USERS) {
117         printf("Database full! Cannot register new members.\n");
118         return NULL;
119     }
120
121     User* new_user = &userDB[total_users];
122     new_user->id = new_id;
123     new_user->is_staff = 0;
124     new_user->book_count = 0;
125
126     printf("\n--- Membership Registration ---\n");
127     printf("Enter your Full Name: ");
128     fgets(new_user->name, sizeof(new_user->name), stdin);
129     stripNewline(new_user->name);
130
131     new_user->membership_start = time(NULL);
132     new_user->membership_expiry = new_user->membership_start + MEMBERSHIP_DURATION;
133
134     total_users++;
135     printf("Success! Membership generated for %s.\n", new_user->name);
136     return new_user;
137 }
138
139 // --- Module 1: O(log n) Binary Search Algorithms ---
140 Book* searchByTitle(char* target) {
141     int left = 0, right = total_books - 1;

```



```

142     while (left <= right) {
143         int mid = left + (right - left) / 2;
144         int cmp = strcmp(bookDB[mid].title, target);
145         if (cmp == 0) return &bookDB[mid];
146         if (cmp < 0) left = mid + 1;
147         else right = mid - 1;
148     }
149     return NULL;
150 }
151
152 Book* searchByAuthor(char* target) {
153     int left = 0, right = total_books - 1;
154     while (left <= right) {
155         int mid = left + (right - left) / 2;
156         int cmp = strcmp(authorIndex[mid]->author, target);
157         if (cmp == 0) return authorIndex[mid];
158         if (cmp < 0) left = mid + 1;
159         else right = mid - 1;
160     }
161     return NULL;
162 }
163
164 // --- Module 1: Member Borrow Logic ---
165 void borrowBookProcess(User* member) {
166     if (member->book_count >= MAX_BORROWED_PER_USER) {
167         printf("Borrow limit reached! Please return a book first.\n");
168         return;
169     }
170
171     int choice;
172     char query[100];
173     Book* foundBook = NULL;
174
175     printf("\nSearch Book By:\n1. Title\n2. Author\nChoice: ");
176     scanf("%d", &choice);
177     getchar(); // Consume newline
178
179     printf("Enter exact search term (Case Sensitive): ");
180     fgets(query, sizeof(query), stdin);
181     stripNewline(query);
182
183     if (choice == 1) foundBook = searchByTitle(query);
184     else if (choice == 2) foundBook = searchByAuthor(query);
185
186     if (foundBook != NULL) {
187         printf("\n--- Book Found ---\n");
188         printf("Title: %s\nAuthor: %s\nGenre: %s\nStock: %d\n", foundBook->title, foundBook->author, foundBook->genre, foundBook->stock);
189
190         if (foundBook->stock > 0) {
191             // Update Book Inventory

```

```

192         foundBook->stock--;
193         foundBook->borrow_count++;
194
195         // Update Member's Borrowed List
196         BorrowedBook* new_borrow = &member->borrowed_books[member->book_count];
197         new_borrow->book_id = foundBook->id;
198         strcpy(new_borrow->title, foundBook->title);
199         new_borrow->issue_time = time(NULL);
200         new_borrow->due_time = new_borrow->issue_time + BOOK_BORROW_PERIOD;
201         member->book_count++;
202
203         printf("Success! You borrowed '%s'.\n", foundBook->title);
204         printf("Please return it within %d seconds to avoid fines.\n", BOOK_BORROW_PERIOD);
205     } else {
206         printf("Sorry, this book is currently out of stock.\n");
207     }
208 } else {
209     printf("Book not found in database.\n");
210 }
211 }
212
213 // --- Member Dashboard (Combined M1 & M2) ---
214 void memberDashboard(User* member) {
215     int choice;
216     while (1) {
217         time_t current_time = time(NULL);
218         double mem_time_left = difftime(member->membership_expiry, current_time);
219
220         printf("\n===== \n");
221         printf("      MEMBER DASHBOARD: %s\n", member->name);
222         printf("===== \n");
223
224         if (mem_time_left <= 0) {
225             printf("!!! YOUR MEMBERSHIP HAS EXPIRED !!!\n");
226             return; // Force logout
227         }
228
229         printf("Membership: ACTIVE (");
230         printTimeRemaining(mem_time_left);
231         printf(")\n\n");
232
233         // Circulation Status
234         double total_fines = 0;
235         printf("--- Circulation Status ---\n");
236         if (member->book_count == 0) {
237             printf("No books currently borrowed.\n");
238         } else {
239             for (int i = 0; i < member->book_count; i++) {
240                 printf("%d. %s\n", i+1, member->borrowed_books[i].title);
241                 double time_until_due = difftime(member->borrowed_books[i].due_time, current_time);

```

```

242
243         if (time_until_due > 0) {
244             printf("    Due in: ");
245             printTimeRemaining(time_until_due);
246             printf("\n");
247         } else {
248             double overdue = -time_until_due;
249             double fine = overdue * FINE_PER_SECOND;
250             total_fines += fine;
251             printf("    STATUS: OVERDUE by %.0f seconds! (Fine: $%.2f)\n", overdue, fine);
252         }
253     }
254     if (total_fines > 0) printf("\n>>> TOTAL UNPAID FINES: $%.2f <<<\n", total_fines);
255 }
256
257 printf("\nOptions:\n1. Search & Borrow a Book\n2. Refresh Circulation Time\n3. Logout\nChoice: ");
258 scanf("%d", &choice);
259 getchar();
260
261 if (choice == 1) borrowBookProcess(member);
262 else if (choice == 3) break;
263 }
264 }
265
266 // --- Staff Dashboard (Module 1) ---
267 void staffDashboard(User* staff) {
268     int choice;
269
270     // Auto-Alerts on Login
271     printf("\n--- System Alerts ---\n");
272     int alerts = 0;
273     for (int i = 0; i < total_books; i++) {
274         if (bookDB[i].stock <= LOW_STOCK_THRESHOLD) {
275             printf("[!] ALERT: '%s' stock is critical (%d remaining)\n", bookDB[i].title, bookDB[i].stock);
276             alerts++;
277         }
278     }
279     if (alerts == 0) printf("Inventory healthy. No low stock alerts.\n");
280
281     while (1) {
282         printf("\n=====");
283         printf("        STAFF DASHBOARD: %s\n", staff->name);
284         printf("=====");
285         printf("1. Update Inventory Stock\n2. Report Analysis (Most Borrowed)\n3. Logout\nChoice: ");
286         scanf("%d", &choice);
287         getchar();
288
289         if (choice == 1) {
290             char query[100];

```

```

291     printf("Enter Book Title to update: ");
292     fgets(query, sizeof(query), stdin);
293     stripNewline(query);
294
295     Book* b = searchByTitle(query);
296     if (b) {
297         printf("Current stock: %d. Enter new stock amount: ", b->stock);
298         scanf("%d", &b->stock);
299         printf("Stock updated!\n");
300         if (b->stock <= LOW_STOCK_THRESHOLD) {
301             printf(">> AUTOMATIC ALERT: Stock for '%s' dropped to critical levels.\n", b->title);
302         }
303     } else {
304         printf("Book not found.\n");
305     }
306 } else if (choice == 2) {
307     printf("\n--- End of Cycle Report Analysis ---\n");
308     int max_idx = 0;
309     for (int i = 1; i < total_books; i++) {
310         if (bookDB[i].borrow_count > bookDB[max_idx].borrow_count) max_idx = i;
311     }
312     printf("Top Borrowed Book:\n");
313     printf("- Title: %s\n- Author: %s\n- Genre: %s\n- Total Circulation: %d times\n",
314           bookDB[max_idx].title, bookDB[max_idx].author, bookDB[max_idx].genre, bookDB[max_idx].borrow_count);
315 } else if (choice == 3) {
316     break;
317 }
318 }
319 }
320
321 // --- Main Execution ---
322 int main() {
323     initSystem();
324     int userId;
325     char response;
326
327     while (1) {
328         printf("\n=====");
329         printf(" Unified Library Management System (Module 1 & 2) \n");
330         printf("=====");
331         printf("Enter ID to Login (101: Member, 999: Staff, 0: Exit): ");
332         scanf("%d", &userId);
333         getchar();
334
335         if (userId == 0) {
336             printf("Shutting down system...\n");
337             break;
338         }
339
340         User* currentUser = findUser(userId);

```

```

342     if (currentUser != NULL) {
343         if (currentUser->is_staff == 1) {
344             staffDashboard(currentUser);
345         } else {
346             memberDashboard(currentUser);
347         }
348     } else {
349         // Module 2 logic: Insist on membership if not found
350         printf("\nID '%d' not found.\n", userId);
351         printf("You must acquire a membership to access the system. Register now? (y/n): ");
352         scanf("%c", &response);
353         getchar();
354
355         if (response == 'y' || response == 'Y') {
356             currentUser = createMembership(userId);
357             if (currentUser != NULL) {
358                 memberDashboard(currentUser);
359             }
360         } else {
361             printf("Access Denied: Membership required.\n");
362         }
363     }
364 }
365 return 0;
366 }

```